

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №9

НАСТРОЙКА СИСТЕМЫ ВИЗУАЛИЗАЦИИ ДАННЫХ ДЛЯ ЗАДАЧ
УПРАВЛЕНИЯ СМАРТ-УСТРОЙСТВАМИ

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков визуализации данных в системе интернета вещей.

2 Задание

Создать систему визуализации данных из базы данных в виде графика. Самостоятельно изучить настройки плагина chart.js и сделать график визуально более привлекательным.

Вариант задания - Умная теплица.

3 Выполнение задания

Для реализации визуализации исторических данных температуры были выполнены следующие шаги. На серверной стороне в файл app.py добавлен новый маршрут /api/temperature_history, который извлекает из коллекции SensorReadings MongoDB последние N записей температуры (по умолчанию 30). При формировании ответа временные метки преобразуются в строковый формат, а значения температуры возвращаются в виде массива чисел. Для доступа к базе данных использован существующий объект database с подключением через pymongo. На рисунке 1 представлен код добавленного маршрута.

```
@app.route("/api/temperature_history") new *
def temperature_history():
    """Возвращает историю температуры для построения графика."""
    limit = request.args.get("limit", default=30, type=int)

    collection = database.db["SensorReadings"]

    cursor = collection.find(
        {"sensor.type": "temperature"},
        {"timestamp": 1, "value": 1, "_id": 0}
    ).sort("timestamp", pymongo.DESCENDING).limit(limit)

    records = list(cursor)[::-1]

    timestamps = []
    values = []
    for rec in records:
        ts = rec["timestamp"]
        if isinstance(ts, datetime):
            timestamps.append(ts.strftime("%Y-%m-%d %H:%M:%S"))
        else:
            timestamps.append(str(ts))
        values.append(rec["value"])

    return jsonify({
        "timestamps": timestamps,
        "values": values
    })
```

Рисунок 1 – Новый маршрут

На клиентской стороне в файл dashboard.html добавлена новая секция, расположенная после всех существующих блоков («Агроном» и «Управление оборудованием»). Для того чтобы секция с графиком занимала всю ширину и располагалась снизу, ей задано CSS-свойство flex: 0 0 100% (поскольку родительский контейнер .container использует display: flex). Внутри секции размещены выпадающий список для выбора количества отображаемых записей (10, 30, 60, 100), кнопка обновления графика и элемент <canvas> с идентификатором temperatureChart. Также в <head> добавлена ссылка на библиотеку Chart.js (рисунок 2 – фрагмент HTML-кода).

```
<section class="card main" style="width: 100%; flex: 0 0 100%; box-sizing: border-box;">
  <h2>Динамика температуры (история)</h2>
  <div style="margin-bottom: 1rem;">
    <label for="history_limit">Показать записей: </label>
    <select id="history_limit">
      <option value="10">10</option>
      <option value="30" selected>30</option>
      <option value="60">60</option>
      <option value="100">100</option>
    </select>
    <button id="refresh_chart" class="btn-primary" style="margin-left: 1rem;">Обновить график</button>
  </div>
  <canvas id="temperatureChart" width="800" height="300" style="width: 100%; height: auto; max-width: 100%;"></canvas>
</section>
```

Рисунок 2 – Секция с графиком

В файл static/js/index.js добавлены новые функции: глобальная переменная temperatureChart, функция loadTemperatureHistory(limit), которая выполняет AJAX-запрос к серверному маршруту и строит линейный график с помощью Chart.js. При создании графика настроены параметры внешнего вида: цвет линии и заливки (в соответствии с цветовой схемой приложения), сглаживание (tension: 0.3), отображение точек, подписи осей («Температура, С» и «Время»), легенда и всплывающие подсказки с форматом значения.

В \$(document).ready организована инициализация графика при загрузке страницы, обработчик нажатия кнопки обновления и (опционально) автоматическое обновление графика каждые 5 секунд (рисунок 3 –JavaScript).

```

$(document).ready(function() {
    let currentLimit = $("#history_limit").val();
    loadTemperatureHistory(currentLimit);

    $("#refresh_chart").on("click", function() {
        let limit = $("#history_limit").val();
        loadTemperatureHistory(limit);
    });

    setInterval(function() {
        let limit = $("#history_limit").val();
        loadTemperatureHistory(limit);
    }, 5000);
});

setInterval(updateDashboard, 1000);
updateDashboard();

```

Рисунок 3 – Построение графика

После запуска MongoDB и Flask-приложения, а также после накопления нескольких записей в коллекции SensorReadings (датчики генерируют и сохраняют значения при каждом изменении) на веб-странице отобразился график температуры. При выборе другого количества записей (например, 10) и нажатии кнопки «Обновить график» диаграмма перестроилась, отобразив соответствующий отрезок истории. При наведении курсора на точки графика появляются всплывающие подсказки с точными значениями температуры и времени. На рисунке 4 представлен скриншот итогового графика.

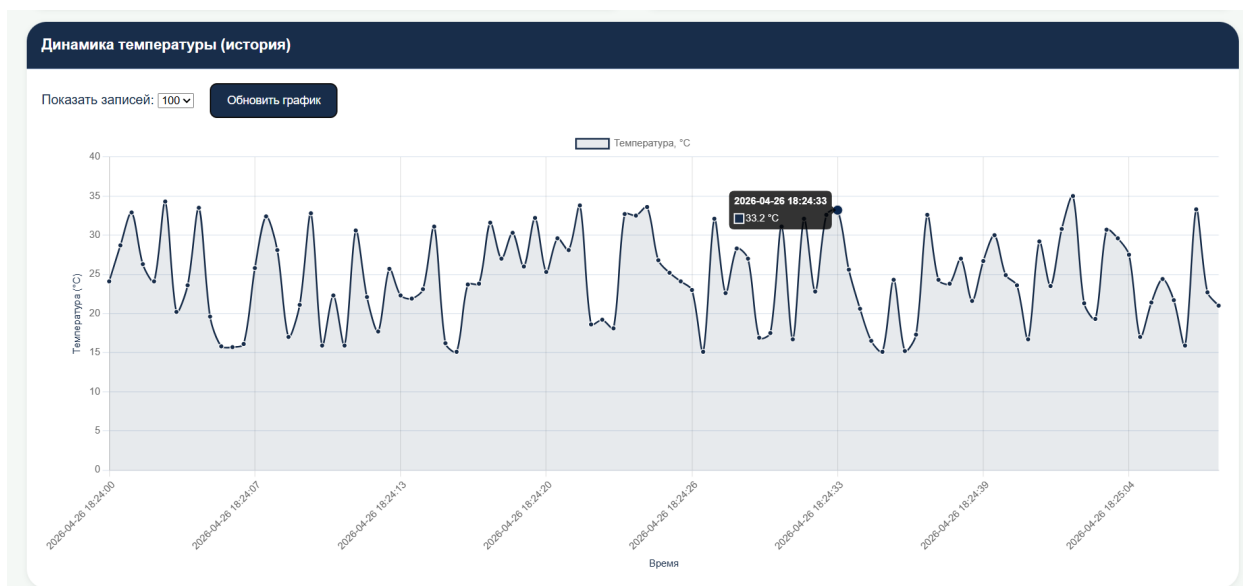


Рисунок 4 – Построенный график

С листингом основных файлов проекта можно ознакомиться в Приложениях к лабораторной работе.

Вывод

В результате выполнения лабораторной работы создана система визуализации данных температуры на основе библиотеки Chart.js. Реализован серверный маршрут для получения исторических данных из MongoDB, клиентская часть отображает линейный график с возможностью выбора глубины истории и ручного обновления. График интегрирован в существующий веб-интерфейс, расположен внизу страницы на всю ширину и визуально согласован с общим дизайном. Полученные навыки визуализации данных могут быть применены для создания панелей мониторинга в реальных IoT-проектах, позволяя анализировать динамику показателей и принимать обоснованные управляющие решения.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

app.py

```
import logging

from flask import Flask, jsonify, render_template, request

from db import Database
from devices import Actuator, MainControlUnit, Sensor
from logger import Logger
import pymongo
from datetime import datetime

app = Flask(__name__)

temperature_sensor = Sensor(1, "Датчик температуры", "temperature", "°C",
"temperature")
humidity_sensor = Sensor(2, "Датчик влажности воздуха", "humidity", "%",
"humidity")
soil_sensor = Sensor(3, "Датчик влажности почвы", "soil_moisture", "%", "soil")
fan = Actuator(101, "Вентилятор", "fan", 120.0)
pump = Actuator(102, "Насос полива", "pump", 80.0)
controller = MainControlUnit(fan_device=fan, pump_device=pump)

database = Database(mongo_uri="mongodb://localhost:27017",
db_name="greenhouse_logs")
db_logger = Logger(database)

last_temperature = 0.0
last_soil = 0.0

temperature_sensor.turn_on()
humidity_sensor.turn_on()
soil_sensor.turn_on()
fan.turn_on()
pump.turn_on()

@app.route("/")
def index():
    return render_template("dashboard.html")
```

```

@app.route("/connect/temperature")
def connect_temperature():
    global last_temperature
    data = temperature_sensor.connect()
    last_temperature = data["value"]
    controller.auto_control(float(last_temperature), float(last_soil))
    db_logger.insert_sensor_reading(
        sensor_id=int(data["id"]),
        sensor_name=str(data["name"]),
        sensor_type=str(data["type"]),
        value=float(data["value"]),
        unit=str(data["unit"]),
        status=bool(data["status"]),
    )
    db_logger.insert_device_event(
        device_id=fan.id,
        device_name=fan.name,
        device_type=fan.type,
        status=bool(fan.status),
        power=float(fan.power),
        event="auto_control",
        details={"reason": "temperature update"},
    )
    db_logger.insert_device_event(
        device_id=pump.id,
        device_name=pump.name,
        device_type=pump.type,
        status=bool(pump.status),
        power=float(pump.power),
        event="auto_control",
        details={"reason": "temperature update"},
    )
    return jsonify(data)

```

```

@app.route("/connect/soil")
def connect_soil():
    global last_soil
    data = soil_sensor.connect()
    last_soil = data["value"]
    controller.auto_control(float(last_temperature), float(last_soil))
    db_logger.insert_sensor_reading(
        sensor_id=int(data["id"]),
        sensor_name=str(data["name"]),
        sensor_type=str(data["type"]),

```

```

        value=float(data["value"]),
        unit=str(data["unit"]),
        status=bool(data["status"]),
    )
    db_logger.insert_device_event(
        device_id=fan.id,
        device_name=fan.name,
        device_type=fan.type,
        status=bool(fan.status),
        power=float(fan.power),
        event="auto_control",
        details={"reason": "soil update"},
    )
    db_logger.insert_device_event(
        device_id=pump.id,
        device_name=pump.name,
        device_type=pump.type,
        status=bool(pump.status),
        power=float(pump.power),
        event="auto_control",
        details={"reason": "soil update"},
    )
    return jsonify(data)

```

```

@app.route("/connect/humidity")
def connect_humidity():
    data = humidity_sensor.connect()
    db_logger.insert_sensor_reading(
        sensor_id=int(data["id"]),
        sensor_name=str(data["name"]),
        sensor_type=str(data["type"]),
        value=float(data["value"]),
        unit=str(data["unit"]),
        status=bool(data["status"]),
    )
    return jsonify(data)

```

```

@app.route("/connect/fan")
def connect_fan():
    data = fan.connect()
    db_logger.insert_device_event(
        device_id=int(data["id"]),
        device_name=str(data["name"]),

```

```

        device_type=str(data["type"]),
        status=bool(data["status"]),
        power=float(data["power"]),
        event="manual_poll",
    )
    return jsonify(data)

```

```

@app.route("/connect/pump")
def connect_pump():
    data = pump.connect()
    db_logger.insert_device_event(
        device_id=int(data["id"]),
        device_name=str(data["name"]),
        device_type=str(data["type"]),
        status=bool(data["status"]),
        power=float(data["power"]),
        event="manual_poll",
    )
    return jsonify(data)

```

```

@app.route("/connect/controller")
def connect_controller():
    return jsonify(controller.connect())

```

```

@app.route("/connect/database")
def connect_database():
    return jsonify(database.connect())

```

```

@app.route("/connect/command", methods=["GET"])
def connect_command():
    """Применить управляющие параметры ко всем вещам."""
    return jsonify(
        {
            "temperature": temperature_sensor.connect(request),
            "humidity": humidity_sensor.connect(request),
            "soil": soil_sensor.connect(request),
            "fan": fan.connect(request),
            "pump": pump.connect(request),
            "controller": controller.connect(request),
            "database": database.connect(request),
        }
    )

```

```

@app.route("/api/temperature_history")
def temperature_history():
    """Возвращает историю температуры для построения графика."""
    limit = request.args.get("limit", default=30, type=int)

    collection = database.db["SensorReadings"]

    cursor = collection.find(
        {"sensor.type": "temperature"},
        {"timestamp": 1, "value": 1, "_id": 0}
    ).sort("timestamp", pymongo.DESCENDING).limit(limit)

    records = list(cursor)[::-1]

    timestamps = []
    values = []
    for rec in records:
        ts = rec["timestamp"]
        if isinstance(ts, datetime):
            timestamps.append(ts.strftime("%Y-%m-%d %H:%M:%S"))
        else:
            timestamps.append(str(ts))
        values.append(rec["value"])

    return jsonify({
        "timestamps": timestamps,
        "values": values
    })

if __name__ == "__main__":
    logging.getLogger("werkzeug").setLevel(logging.ERROR)
    app.run(debug=True)

```

ПРИЛОЖЕНИЕ В

devices.py

```

import random
import re
from abc import ABC, abstractmethod
from typing import Dict, List, Optional, Union

from werkzeug.wrappers import Request

class Device(ABC):
    def __init__(self, device_id: int, name: str, status: bool = False):
        self.id = device_id

```

```
self.name = name
self.status = status
```

```
def turn_on(self):
    self.status = True
    print(f'[LOG] {self.name} (ID: {self.id}) включён")
```

```
def turn_off(self):
    self.status = False
    print(f'[LOG] {self.name} (ID: {self.id}) выключен")
```

```
def get_status(self) -> bool:
    print(f'[LOG] Запрос статуса {self.name} (ID: {self.id}) -> {self.status}")
    return self.status
```

```
@abstractmethod
def connect(
    self, request: Optional[Request] = None
) -> Dict[str, Union[int, float, str, bool]]:
    """Мониторинг (request is None) или применение команд (передан
request)."""
```

```
class Sensor(Device):
    def __init__(
        self,
        device_id: int,
        name: str,
        sensor_type: str,
        unit: str,
        control_prefix: str,
    ):
        super().__init__(device_id, name)
        self.type = sensor_type
        self.unit = unit
        self.control_prefix = control_prefix
        self.value: float = 0.0
```

```
def measure(self):
    # Генерация случайных значений
    if self.type == "temperature":
        self.value = round(random.uniform(15.0, 35.0), 1)
    elif self.type == "humidity":
        self.value = round(random.uniform(30.0, 80.0), 1)
    elif self.type == "soil_moisture":
```

```

        self.value = round(random.uniform(20.0, 60.0), 1)
    else:
        self.value = round(random.uniform(0, 100), 1)
    print(
        f"[LOG] Датчик {self.name} (ID: {self.id}) измерил: {self.value}
{self.unit}"
    )
    return self.value

def calibrate(self, value: float):
    self.value = value
    print(
        f"[LOG] Датчик {self.name} (ID: {self.id}) откалиброван на значение
{value} {self.unit}"
    )

def _apply_control(self, request: Request) -> None:
    p = self.control_prefix
    power_raw = request.args.get(f"{p}_power", "").strip().lower()
    if power_raw:
        # Допустимо on, off, 1, 0, true, false
        if re.match(r"^(on|off|1|0|true|false)$", power_raw):
            if power_raw in ("on", "1", "true"):
                self.turn_on()
            else:
                self.turn_off()
        else:
            print(
                f"[LOG] Ошибка: недопустимое значение power '{power_raw}' для
{self.name}. Допустимо: on/off/1/0/true/false"
            )

    raw_cal = request.args.get(f"{p}_calibrate", "").strip()
    if raw_cal:
        try:
            # Заменяем запятую на точку и преобразуем в float
            value = float(raw_cal.replace(",", "."))
            self.calibrate(value)
        except ValueError:
            print(
                f"[LOG] Ошибка: калибровка {self.name} – некорректное число
'{raw_cal}'"
            )

def connect(

```

```

        self, request: Optional[Request] = None
    ) -> Dict[str, Union[int, float, str, bool]]:
        if request is not None:
            self._apply_control(request)
            print(
                f"[LOG] Управляющая команда для датчика {self.name}: значение
{self.value} {self.unit}, статус {self.status}"
            )
        else:
            # Без команд выполняем измерение
            self.measure()
            print(
                f"[LOG] Опрос датчика {self.name}: текущее значение {self.value}
{self.unit}"
            )
        return {
            "id": self.id,
            "name": self.name,
            "type": self.type,
            "unit": self.unit,
            "value": self.value,
            "status": self.status,
        }

```

```

class Actuator(Device):
    def __init__(
        self, device_id: int, name: str, actuator_type: str, power: float = 0.0
    ):
        super().__init__(device_id, name)
        self.type = actuator_type
        self.power = power
        self.control_prefix = actuator_type

    def apply_action(self, action: str):
        print(
            f"[LOG] Исполнительное устройство {self.name} (ID: {self.id})
выполняет действие: {action}"
        )

    def set_power(self, level: float):
        self.power = level
        print(
            f"[LOG] Мощность устройства {self.name} (ID: {self.id}) установлена
на {level} Вт"
        )

```

```

)

def _apply_control(self, request: Request) -> None:
    p = self.control_prefix

    power_kw = request.args.get(f'{p}_power_cmd', "").strip()
    if power_kw:
        try:
            self.set_power(float(power_kw.replace(",", ".")))
        except ValueError:
            print(
                f'[LOG] Ошибка: мощность {self.name} – некорректное число '
                f'{power_kw}'
            )

    action = request.args.get(f'{p}_action', "").strip()
    if action:
        if re.match(r"^[a-zA-Za-яA-Я0-9_\-]+$", action):
            self.apply_action(action)
        else:
            print(
                f'[LOG] Ошибка: действие '{action}' для {self.name} содержит '
                f'недопустимые символы. Разрешены буквы, цифры, '_', '-'
            )

    st = request.args.get(f'{p}_power_state', "").strip().lower()
    if st:
        if re.match(r"^(on|off|1|0|true|false)$", st):
            if st in ("on", "1", "true"):
                self.turn_on()
            else:
                self.turn_off()
        else:
            print(
                f'[LOG] Ошибка: неверное состояние питания '{st}' для '
                f'{self.name}'
            )

    def connect(
        self, request: Optional[Request] = None
    ) -> Dict[str, Union[int, float, str, bool]]:
        if request is not None:
            self._apply_control(request)
        print(
            f'[LOG] Управление {self.name}: мощность {self.power} Вт, статус

```

```

{self.status}"
    )
    else:
        print(
            f"[LOG] Опрос исполнительного устройства {self.name}:
{self.power} Вт, статус: {self.status}"
        )
    return {
        "id": self.id,
        "name": self.name,
        "type": self.type,
        "power": self.power,
        "status": self.status,
    }

```

```

class MainControlUnit:
    def __init__(self, fan_device=None, pump_device=None):
        self.mode = "auto"
        self.schedule: Dict[str, str] = {}
        self.alerts: List[str] = []
        self.fan = fan_device
        self.pump = pump_device

    def process_data(self, sensor_data: Dict[int, float]):
        print(f"[LOG] Контроллер обрабатывает данные с датчиков:
{sensor_data}")

    def send_command(self, actuator: Actuator, command: str):
        print(
            f"[LOG] Контроллер отправляет команду '{command}' устройству
{actuator.name} (ID: {actuator.id})"
        )
        actuator.apply_action(command)

    def check_alerts(self):
        print("[LOG] Контроллер проверяет наличие тревог...")
        if self.alerts:
            print(f"[LOG] Активные тревоги: {self.alerts}")
        else:
            print("[LOG] Тревог нет")

    def _apply_control(self, request: Request) -> None:
        mode = request.args.get("controller_mode", "").strip().lower()
        if mode:

```

```

        if re.match(r"^(auto|manual)$", mode):
            self.mode = mode
            print(f"[LOG] Контроллер: режим установлен в {self.mode}")
        else:
            print(
                f"[LOG] Ошибка: неверный режим '{mode}'. Допустимо: auto /
manual"
            )

    clear = request.args.get("controller_clear_alerts", "").strip().lower()
    if clear:
        if re.match(r"^(1|true|yes|on)$", clear):
            self.alerts = []
            print("[LOG] Контроллер: список тревог очищен")
        else:
            print(
                f"[LOG] Ошибка: неверное значение clear_alerts '{clear}'.
Игнорируем."
            )

    def connect(
        self, request: Optional[Request] = None
    ) -> Dict[str, Union[str, int, List[str]]]:
        if request is not None:
            self._apply_control(request)
        print(
            f"[LOG] Подключение к контроллеру выполнено, режим: {self.mode},
количество тревог: {len(self.alerts)}"
        )
        return {
            "mode": self.mode,
            "alerts_count": len(self.alerts),
            "alerts": list(self.alerts),
        }

    def auto_control(self, temperature: float, soil_moisture: float):
        """Автоматическое управление на основе показаний датчиков"""
        if self.mode != "auto":
            print("[LOG] Автоматика отключена (ручной режим)")
            return

        # Управление вентилятором по температуре
        if self.fan:
            if temperature > 28.0 and not self.fan.status:
                self.fan.turn_on()

```

```

        self.alerts.append(f"Авто: вентилятор включён при {temperature}°C")
        print(
            f"[LOG] Автоматика: включён вентилятор (температура
{temperature}°C)"
        )
        elif temperature <= 28.0 and self.fan.status:
            self.fan.turn_off()
            self.alerts.append(f"Авто: вентилятор выключен при
{temperature}°C")
            print(
                f"[LOG] Автоматика: выключен вентилятор (температура
{temperature}°C)"
            )

# Управление насосом по влажности почвы
if self.pump:
    if soil_moisture < 30.0 and not self.pump.status:
        self.pump.turn_on()
        self.alerts.append(
            f"Авто: насос включён (влажность почвы {soil_moisture}%)"
        )
        print(
            f"[LOG] Автоматика: включён насос (влажность почвы
{soil_moisture}%)"
        )
    elif soil_moisture >= 30.0 and self.pump.status:
        self.pump.turn_off()
        self.alerts.append(
            f"Авто: насос выключен (влажность почвы {soil_moisture}%)"
        )
        print(
            f"[LOG] Автоматика: выключен насос (влажность почвы
{soil_moisture}%)"
        )

```

ПРИЛОЖЕНИЕ С

index.js

```

let units = {
    temperature: "",
    humidity: "",
    soil_moisture: "",
};

function getTemperatureData() {
    $.ajax({
        type: "GET",

```

```

        url: "/connect/temperature",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#temperature_value").val(response.value + " " + response.unit);
            units.temperature = response.unit || "";
        },
    });
}

function getHumidityData() {
    $.ajax({
        type: "GET",
        url: "/connect/humidity",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#humidity_value").val(response.value + " " + response.unit);
            units.humidity = response.unit || "";
        },
    });
}

function getSoilData() {
    $.ajax({
        type: "GET",
        url: "/connect/soil",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#soil_value").val(response.value + " " + response.unit);
            units.soil_moisture = response.unit || "";
        },
    });
}

function getFanData() {
    $.ajax({
        type: "GET",
        url: "/connect/fan",
        dataType: "json",
        contentType: "application/json",
        data: {},
    });
}

```

```

        success: function (response) {
            $("#fan_power").val(
                response.power + " Вт; статус: " + response.status
            );
        },
    });
}

```

```

function getPumpData() {
    $.ajax({
        type: "GET",
        url: "/connect/pump",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#pump_power").val(
                response.power + " Вт; статус: " + response.status
            );
        },
    });
}

```

```

function getControllerData() {
    $.ajax({
        type: "GET",
        url: "/connect/controller",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#controller_mode").val("Режим: " + response.mode);
            $("#controller_alerts").val("Тревоги: " + response.alerts_count);
        },
    });
}

```

```

function getDatabaseData() {
    function formatValue(number, unit, fractionDigits = 1) {
        if (typeof number !== "number" || Number.isNaN(number)) return "—";
        return number.toFixed(fractionDigits) + " " + (unit || "");
    }

    $.ajax({
        type: "GET",

```

```

url: "/connect/database",
dataType: "json",
contentType: "application/json",
data: {},
success: function (response) {
    $("#db_connection").val("Соединение: " + response.connection);
    if (response.db_name)
        $("#db_name").val("MongoDB база: " + response.db_name);
    else $("#db_name").val("MongoDB база: —");

    if (typeof response.records_total !== "undefined")
        $("#db_records").val("Записей всего: " + response.records_total);
    else $("#db_records").val("Записей всего: —");

    if (response.collections) {
        const entries = Object.entries(response.collections);
        entries.sort((a, b) => (b[1] || 0) - (a[1] || 0));
        const lines = entries.map(([k, v]) => `${k}: ${v}`);
        $("#db_collections").val(lines.join("\n") || "Коллекций пока нет");
    } else $("#db_collections").val("Коллекций: —");

    const analytics = response.analytics || {};
    const temp = analytics.temperature || {};
    const humidity = analytics.humidity || {};
    const soil_moisture = analytics.soil_moisture || {};

    $("#temperature_avg").val(
        temp.error ? "ошибка" : formatValue(temp.avg, units.temperature)
    );
    $("#temperature_max").val(
        temp.error ? "ошибка" : formatValue(temp.max, units.temperature)
    );

    $("#humidity_avg").val(
        humidity.error ? "ошибка" : formatValue(humidity.avg, units.humidity)
    );
    $("#humidity_max").val(
        humidity.error ? "ошибка" : formatValue(humidity.max, units.humidity)
    );

    $("#soil_avg").val(
        soil_moisture.error
            ? "ошибка"
            : formatValue(soil_moisture.avg, units.soil_moisture)
    );
}

```

```

        $("#soil_max").val(
            soil_moisture.error
            ? "ошибка"
            : formatValue(soil_moisture.max, units.soil_moisture)
        );
    },
});
}

```

```

function updateDashboard() {
    getTemperatureData();
    getHumidityData();
    getSoilData();
    getFanData();
    getPumpData();
    getControllerData();
    getDatabaseData();
}

```

```

function collectCommandPayload() {
    return {
        temperature_power: $("#temperature_power").val(),
        temperature_calibrate: $("#temperature_calibrate").val(),
        humidity_power: $("#humidity_power").val(),
        humidity_calibrate: $("#humidity_calibrate").val(),
        soil_power: $("#soil_power").val(),
        soil_calibrate: $("#soil_calibrate").val(),
        fan_power_cmd: $("#fan_power_cmd").val(),
        fan_power_state: $("#fan_power_state").val(),
        fan_action: $("#fan_action").val(),
        pump_power_cmd: $("#pump_power_cmd").val(),
        pump_power_state: $("#pump_power_state").val(),
        pump_action: $("#pump_action").val(),
        controller_mode: $("#controller_mode_cmd").val(),
        controller_clear_alerts: $("#controller_clear_alerts").is(":checked")
            ? "1"
            : "",
        database_command: $("#database_command").val(),
    };
}

```

```

function sendCommands() {
    $.ajax({
        type: "GET",
        url: "/connect/command",
    });
}

```

```

        dataType: "json",
        contentType: "application/json",
        data: collectCommandPayload(),
        success: function (response) {
            $("#command_result").val(JSON.stringify(response, null, 2));
        },
    });
}

$("#send_commands").on("click", sendCommands);

setInterval(updateDashboard, 1000);
updateDashboard();

let temperatureChart = null;

function loadTemperatureHistory(limit = 30) {
    $.ajax({
        type: "GET",
        url: `/api/temperature_history?limit=${limit}`,
        dataType: "json",
        success: function(data) {
            const ctx = document.getElementById('temperatureChart').getContext('2d');
            if (temperatureChart) {
                temperatureChart.destroy();
            }
            temperatureChart = new Chart(ctx, {
                type: 'line',
                data: {
                    labels: data.timestamps,
                    datasets: [{
                        label: 'Температура, °C',
                        data: data.values,
                        borderColor: '#182d4a',
                        backgroundColor: 'rgba(24, 45, 74, 0.1)',
                        borderWidth: 2,
                        tension: 0.3,
                        fill: true,
                        pointBackgroundColor: '#182d4a',
                        pointRadius: 3,
                        pointHoverRadius: 6,
                        pointBorderColor: '#fff',
                        pointBorderWidth: 1
                    }]
                }
            });
        },
    });
}

```

```

options: {
    responsive: true,
    maintainAspectRatio: true,
    interaction: { mode: 'index', intersect: false },
    plugins: {
        legend: { position: 'top' },
        tooltip: {
            callbacks: {
                label: function(context) {
                    return `${context.raw} °C`;
                }
            }
        }
    },
    scales: {
        y: {
            title: { display: true, text: 'Температура (°C)', color: '#182d4a' },
            min: 0,
            max: 40,
            grid: { color: '#e0e7f0' }
        },
        x: {
            title: { display: true, text: 'Время', color: '#182d4a' },
            ticks: {
                maxRotation: 45,
                autoSkip: true,
                maxTicksLimit: 8
            }
        }
    }
}
});
},
error: function(err) {
    console.error("Ошибка загрузки истории температуры:", err);
}
});
}

```

```

$(document).ready(function() {
    let currentLimit = $("#history_limit").val();
    loadTemperatureHistory(currentLimit);

    $("#refresh_chart").on("click", function() {
        let limit = $("#history_limit").val();
    });
});

```

```
    loadTemperatureHistory(limit);
  });

  setInterval(function() {
    let limit = $("#history_limit").val();
    loadTemperatureHistory(limit);
  }, 5000);
});

setInterval(updateDashboard, 1000);
updateDashboard();
```